

Q1 (a)

Difference between data mining and machine learning

Data mining is discovering patterns and relationships in LARGE datasets, IT REFERS TO WHOLE PROCESS.

Machine learning is the subset/step of data mining that focuses on training models to make predictions or decisions.

(b) Discuss distinction between interpretability and explainability in the context of machine learning models and the connection with model performance.

Interpretability is understating how a model makes predictions while explainability is presenting a rational explanation providing human understandable reasons.

The connection with model performance is a trade-off between Interpretability and explainability depending on model complexity.

(c) Define the different between supervised and unsupervised learning and give examples of each.

Supervised learning involve input and output labels and a model trained on data with known outcomes learns to recognised from input features like a picture of a car or a sample of spam email that it filers into the spam folder when is arrives into the recipient inbox.

Unsupervised learning had no output labelling and the code must search the dataset for patterns. It is used for customer sentiment analysis where customer provide product reviews and the marketing want to undertake sentiment analysis.

Q 2 (a) Explain the apriori algorithm including the terms confidence, support and lift.

This algorithm is used by stores to undertake **associate rule mining** market basket analysis to understand their customer transaction in and inform consumer behaviour. The classic consumer items are bread, milk, butter, beer and diapers. They begin by taking the average number of items in a basket and construct 1, 2, 3 items baskets etc.

For confidence, **measures how often a rule is TRUE**

For support, **measures the frequency of an itemset**

For lift **quantifies the rule relative to a random event.**

(b) Discuss the difference between filter and wrapper methods including when it is best to use them.

A filter method **selects features** using a statistical technique like correlation analysis to search through a dataset.

A wrapper method **uses model performance as a criterion for feature selection.**

Wrapper methods are computationally expensive.

LEARN BY HEART

(c) Give a short explanation of random forest, k nearest neighbour, naïve bayes and support vector machine.

Random forest: **An ensemble learning method that combines multiple decision trees** for robust prediction, trained on subsets of data and features.

K nearest neighbour (KNN): An **instance-based classification algorithm that classifies data point based on majority class of k nearest neighbours, using** user defined parameter K and a distance metric.

Naïve bayes: A **probabilistic classifier that assumes feature independence**, calculating class probabilities based on **feature methods and prior probabilities.**

Support vector machine: **A supervised learning algorithm that finds optimal hyperplane to maximises margin between classes**, capable of handling binary and **multiclass tasks through kernel functions.**

3 (a)

Euclidean distance d_1 between two points (x_1, y_1) and (x_2, y_2) where $d_1 = ((y_2 - y_1)^2 + (x_2 - x_1)^2)^{0.5}$

SAX where time series is mapped to letters allowing data compression and finding similarities

What SAX does (in one sentence)

SAX converts a numeric time series into a **short string of symbols** by:

1. Normalizing the data
 2. Reducing dimensionality with **PAA**
 3. Discretizing values using statistically defined **breakpoints**
-

Step 0: Original time series

Suppose we have the following time series of length 12:

T
 $= [2, 4, \quad, 3, \quad, 5$

Our goal is to turn this into a symbolic string like "abccdd".

Step 1: Z-normalization

SAX **requires z-normalization** (mean = 0, std = 1).

1.1 Compute mean and standard deviation

- Mean:

$$\mu = \frac{2 + 4 + 3 + 5 + 6 + 8 + 7 + 9 + 10 + 8 + 6 + 5}{12} = 6.08$$

- Standard deviation:

$$\sigma \approx 2.41$$

1.2 Normalize each value

$$z_i = \frac{x_i - \mu}{\sigma}$$

This gives approximately:

$$Z = [-1.69, \dots, -0.86,$$

Step 2: Piecewise Aggregate Approximation (PAA)

We now **reduce dimensionality**.

2.1 Choose number of segments

Let:

- Original length = 12
- PAA segments = **6**

Each segment contains 2 points.

2.2 Average each segment

Segment	Values	Mean
1	-1.69, -0.86	-1.28
2	-1.28, -0.45	-0.87
3	-0.03, 0.80	0.39
4	0.38, 1.21	0.80
5	1.63, 0.80	1.22
6	-0.03, -0.45	-0.24

So the PAA representation is:

$$PAA = [-1.28, \dots, -0.87, 0$$

Step 3: Discretization into symbols

3.1 Choose alphabet size

Let alphabet size = **4**

Symbols:

$$\{a, b, c, d\}$$

3.2 SAX breakpoints (Gaussian)

For alphabet size = 4, breakpoints are fixed:

Symbol Range

a	$(-\infty, -0.67)$
b	$[-0.67, 0)$
c	$[0, 0.67)$
d	$[0.67, \infty)$

3.3 Map PAA values to symbols

PAA value Symbol

-1.28	a
-0.87	a
0.39	c
0.80	d
1.22	d
-0.24	b

Final SAX representation

"aacddb"

This short string is the **symbolic representation** of the original numeric time series.

Why SAX is useful

- **Dimensionality reduction** ($12 \rightarrow 6 \rightarrow \text{symbols}$)
- **Noise tolerance**

- Enables:
 - Fast similarity search
 - Indexing
 - Pattern discovery
 - Anomaly detection

Key parameters to remember

Parameter	Meaning
w	Number of PAA segments
α	Alphabet size
Z-normalization	Required
Breakpoints	Based on Gaussian distribution

Dynamic Time Warping (DTW) is a technique used to measure similarity between two time series that may be **out of phase in time** (stretched, compressed, or shifted).

Below is a **clear explanation with a worked example**, intuition, and key formulas.

1. Why DTW is needed (intuition)

Euclidean distance compares points **index-by-index**:

$$\left(\begin{array}{cccc} x_1 & \leftrightarrow & y_1, & \\ & & & x_2 \\ & \leftrightarrow & y_2, & \\ & & & \dots \end{array} \right)$$

This fails when:

- One series is slower or faster
- Peaks occur at slightly different times

DTW **warps the time axis** to align similar shapes.

2. Simple motivating example

Consider two sequences:

X
 $= [1, 2 \quad \quad \quad , 3 \quad \quad \quad , 4$
 Y
 $= [1, 1 \quad \quad \quad , 2 \quad \quad \quad , 3$

Same shape, but **Y is stretched**.

- Euclidean distance $\rightarrow$ large
- DTW $\rightarrow$ small

3. Core idea of DTW

DTW finds an **optimal alignment path** through a cost matrix that:

1. Starts at (1,1)
2. Ends at (n,m)
3. Moves only:
 - Right
 - Down
 - Diagonal
4. Minimizes total alignment cost

4. Step-by-step DTW algorithm

Step 1: Construct cost matrix

For each pair of points:

$$D(i,j) = |x_i - y_j|$$

Example:

X\Y 1 1 2 3 4 5 5

1 0 0 1 2 3 4 4

2 1 1 0 1 2 3 3

X\Y 1 1 2 3 4 5 5

3 2 2 1 0 1 2 2

4 3 3 2 1 0 1 1

5 4 4 3 2 1 0 0

Step 2: Build cumulative cost matrix

$$C(i, j) = D(i, j) + \min \begin{cases} C(i-1, j) \\ C(i, j-1) \\ C(i-1, j-1) \end{cases}$$

This enforces monotonic warping.

Step 3: Find optimal warping path

- Start at bottom-right cell
 - Trace back to (1,1) following minimal cost
 - This path defines how time points align
-

5. Final DTW distance

$$\text{DTW}(X, Y) = C(n, m)$$

- Small DTW distance $\Rightarrow$ similar shape
 - Zero distance $\Rightarrow$ identical (possibly warped)
-

6. Visual intuition (warping path)

Y $\rightarrow$ 1 1 2 3 4 5 5

X $\downarrow$

1 * *

2 *

3 *

4 *

5 * *

Multiple Y points align with single X points and vice versa.

7. Key properties of DTW

Advantages

- ✓ Handles time shifts and speed changes
- ✓ Works for unequal-length series
- ✓ Shape-based similarity

Disadvantages

- ✗ Computationally expensive: $O(nm)$
- ✗ Sensitive to noise
- ✗ Can over-warp if unconstrained

Allowing for alignment and similarity very important for speech recognition and chat bots for first point on contact and language translation.

Can be applied to time series using clustering and k means

DTW in one sentence

DTW measures similarity between time series by optimally warping time to align similar shapes.

(b) word2vec and bag of words

Below is a **clear, side-by-side explanation** of **Bag of Words (BoW)** and **Word2Vec**, with intuition, examples, and key differences.

1. Bag of Words (BoW)

Concept

Bag of Words represents text by **counting word occurrences**, ignoring:

- Grammar
- Word order
- Context

Each document becomes a **vector of word frequencies**.

“Bag” means words are treated independently, like items in a bag.

Example: Bag of Words

Text corpus

D1: "I like machine learning"

D2: "I like deep learning"

Step 1: Build vocabulary

Unique words:

["I", "like", "machine", "learning", "deep"]

Step 2: Vectorize documents

Word I like machine learning deep

D1 1 1 1 1 0

D2 1 1 0 1 1

So:

- D1 → [1, 1, 1, 1, 0]
- D2 → [1, 1, 0, 1, 1]

Variants of BoW

- **Binary BoW** → 0 or 1 (presence/absence)
 - **Count BoW** → raw frequency
 - **TF-IDF** → weighted by importance
-

Advantages of BoW

- ✓ Simple and fast
- ✓ Easy to interpret
- ✓ Works well for many classification tasks

Limitations of BoW

- ✗ Ignores word order
 - ✗ Ignores meaning/context
 - ✗ Large sparse vectors
-

2. Word2Vec

Concept

Word2Vec represents each word as a **dense vector** learned from context.

Words that appear in **similar contexts have similar vectors**.

Instead of counting words, Word2Vec **learns meaning from usage**.

How Word2Vec works (intuition)

Given a sentence:

"The cat sits on the mat"

Word2Vec learns that:

- "cat" is related to "dog"
- "king - man + woman $\approx$ queen"

This **semantic knowledge** comes from **context windows**.

Two Word2Vec models

1. **CBOW (Continuous Bag of Words)**
Predicts a **word** from its **context**
 2. **Skip-gram**
Predicts **context** words from a **target word**
-

Example: Word2Vec

Training sentence

"I like machine learning"

Assume embedding size = 3.

After training, vectors might look like:

Word	Vector
------	--------

I	[0.2, 0.1, 0.3]
---	-----------------

like	[0.6, 0.4, 0.5]
------	-----------------

machine	[0.9, 0.1, 0.2]
---------	-----------------

learning	[0.85, 0.15, 0.25]
----------	--------------------

deep	[0.88, 0.12, 0.28]
------	--------------------

Notice:

- "machine" $\approx$ "deep" $\approx$ "learning"
- Vectors are **dense and low-dimensional**

Using Word2Vec for documents

To represent a sentence/document:

- Average word vectors
- Or **use weighted average (TF-IDF + Word2Vec)**

Example:

"I like deep learning"

Document vector $\approx$ mean(I, like, deep, learning)

Advantages of Word2Vec

- ✓ Captures semantic meaning
- ✓ Dense, low-dimensional vectors
- ✓ Handles synonyms well

Limitations of Word2Vec

- ✗ Requires large data to train well
 - ✗ Static embeddings (same vector for every context)
 - ✗ Less interpretable
-

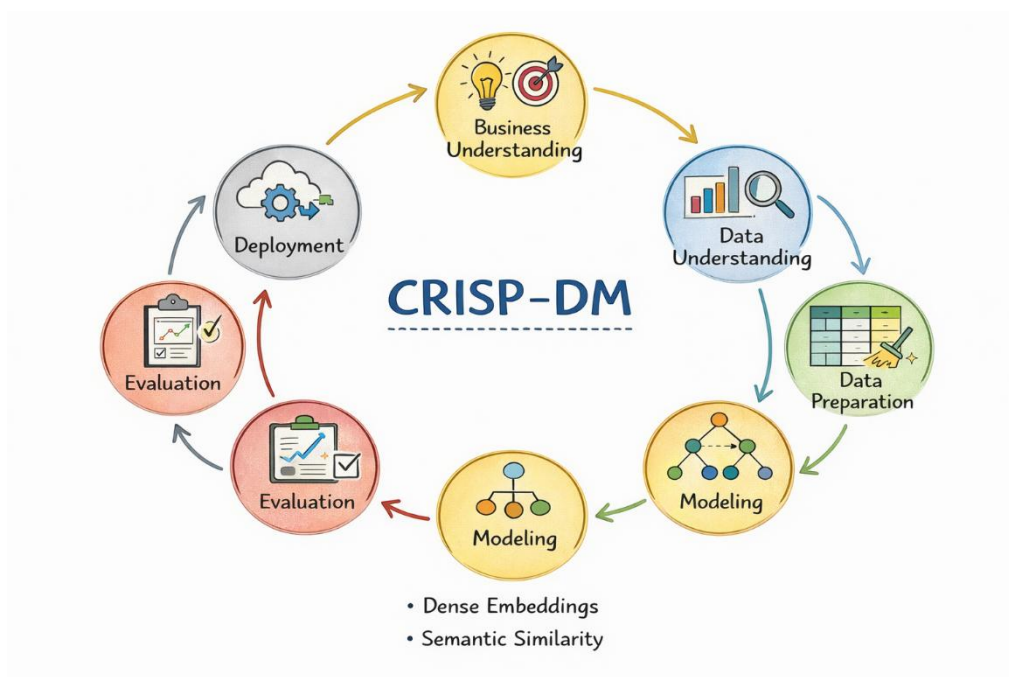
3. Bag of Words vs Word2Vec

Feature	Bag of Words	Word2Vec
Representation	Word counts	Word embeddings
Word order	Ignored	Indirectly captured
Semantics	✗	✓
Vector size	Vocabulary size	Fixed (e.g., 100–300)
Sparsity	Sparse	Dense
Training	None	Neural network

4. Simple analogy

- **Bag of Words:**
“How many times does each word appear?”
- **Word2Vec:**
“What does this word *mean* based on its neighbours?”

(c) CRISP-DM steps



(d) Discuss concept reinforcement learning and its relationship to Markov Decision Processes.

1. What is Reinforcement Learning?

Reinforcement Learning is a learning paradigm where an **agent** learns to make decisions by **interacting with an environment** in order to **maximize cumulative reward**.

Instead of being told the correct action, the agent:

- Tries actions
- Observes consequences
- Receives rewards or penalties
- Improves its behavior over time

Core idea

Learn by trial and error using feedback from the environment.

2. Key components of Reinforcement Learning

An RL problem is defined by:

Component	Description
Agent	Learner/decision-maker
Environment	Everything the agent interacts with
State (s)	Current situation of the agent
Action (a)	Choices available to the agent
Reward (r)	Scalar feedback signal
Policy (π)	Strategy mapping states $\rightarrow$ actions

3. What is a Markov Decision Process (MDP)?

A **Markov Decision Process** is a **mathematical framework** for modeling sequential decision-making problems where outcomes are partly random and partly under the agent's control.

An MDP is defined as a 5-tuple:

$$(S, A, P, R, \gamma)$$

Element	Meaning
S	Set of states
A	Set of actions
P(s' s, a)	Transition probability
R(s, a)	Reward function
γ	Discount factor

4. The Markov property (crucial link)

The defining assumption of an MDP is the **Markov property**:

$$P(s_{t+1} | s_t, a_t, s_{t-1}, a_{t-1}, \dots) = P(s_{t+1} | s_t, a_t)$$

The future depends only on the present state and action, not the past.

This makes planning and learning tractable.

5. Relationship between RL and MDPs

Key relationship

Reinforcement Learning is the solution approach;
Markov Decision Processes are the problem formulation.

- **MDP**: Defines *what* the environment is
 - **RL**: Defines *how* the agent learns to act in it
-

Two cases

1. MDP known → Planning

If *P* and *R* are known:

- Use **Dynamic Programming**
- Examples: Value Iteration, Policy Iteration

2. MDP unknown → Reinforcement Learning

If P and/or R are unknown:

- Learn by interaction
 - Examples: Q-learning, SARSA, Policy Gradients
-

6. Value functions in MDPs and RL

State-value function

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s \right]$$

Action-value function

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s, a_0 = a \right]$$

These quantify **how good** a state or action is.

7. Bellman equations (the bridge)

Bellman equations describe **recursive structure** in MDPs:

Bellman expectation equation

$$V^\pi(s) = \sum_a \pi(a \mid s) \left[R(s, a) + \gamma \sum_{s'} P(s' \mid s, a) V^\pi(s') \right]$$

Bellman optimality equation

$$V^*(s) = \max_a \left[R(s, a) + \gamma \sum_{s'} P(s' \mid s, a) V^*(s') \right]$$

RL algorithms approximate these equations **without knowing** P .

8. Example: Gridworld

- States: grid cells
- Actions: up, down, left, right
- Reward: -1 per step, $+10$ at goal
- Transitions: stochastic

This is an **MDP**.

Using RL (e.g., Q-learning), the agent:

- Explores the grid
 - Learns Q-values
 - Discovers the shortest path to the goal
-

9. Extensions beyond basic MDPs

Model	When used
POMDP	Partial observability
Semi-MDP	Variable-duration actions
Continuous MDPs	Continuous states/actions
Multi-agent MDPs	Multiple agents

10. Why MDPs matter in RL

MDPs provide:

- Theoretical guarantees
- Convergence proofs
- A unifying language for RL algorithms

Almost all classical RL theory assumes the environment is an **MDP**.

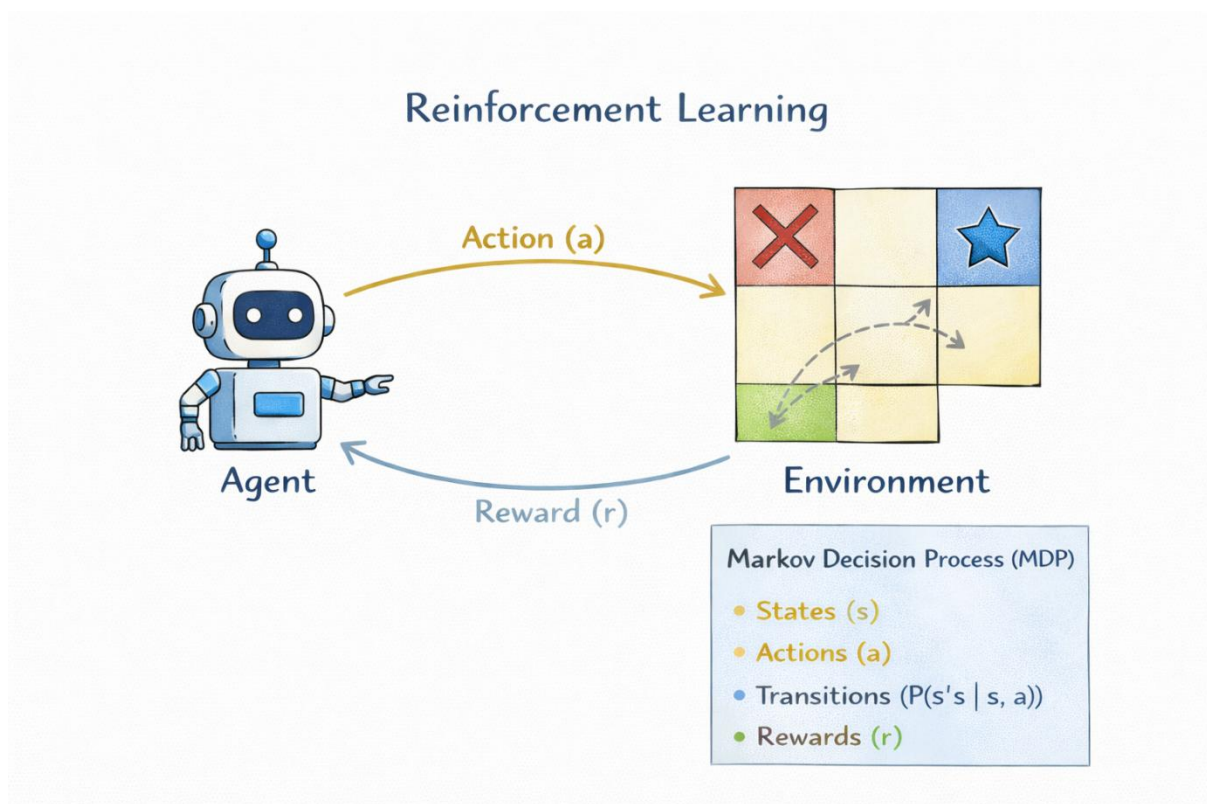
11. Summary (exam-ready)

- **Reinforcement Learning** is a learning paradigm where agents learn optimal behavior via interaction and reward.

- **Markov Decision Processes** provide the mathematical framework for modeling RL environments.
 - RL algorithms aim to solve MDPs **when the model is unknown**.
 - The **Markov property** enables efficient learning and planning.
-

One-line takeaway

Reinforcement Learning learns policies; Markov Decision Processes define the world in which those policies act.



(e) Give short definitions related to explainability: accountability Trustworthiness, confidence, causality, fairness ethical decision

Accountability

The ability to **identify who is responsible** for an AI system's decisions, outcomes, and impacts, including mechanisms for oversight and redress.

Trustworthiness

The degree to which an AI system is **reliable, transparent, safe, and aligned with human values**, enabling users to confidently rely on its outputs.

Confidence

A measure of how **certain an AI system is about its predictions or decisions**, often expressed probabilistically (e.g., prediction confidence or uncertainty).

Causality

The extent to which an AI model captures or explains **cause-and-effect relationships**, rather than mere correlations, between inputs and outcomes.

Fairness

The property of an AI system to **avoid unjustified bias or discrimination** and to treat individuals or groups equitably across sensitive attributes.

Ethical Decision-Making

The process by which AI systems (or their designers) make choices that **respect moral principles**, such as beneficence, non-maleficence, autonomy, and justice.

One-line summary

Explainable AI supports **accountability**, builds **trustworthiness**, communicates **confidence**, reveals **causality**, ensures **fairness**, and enables **ethical decisions**.